

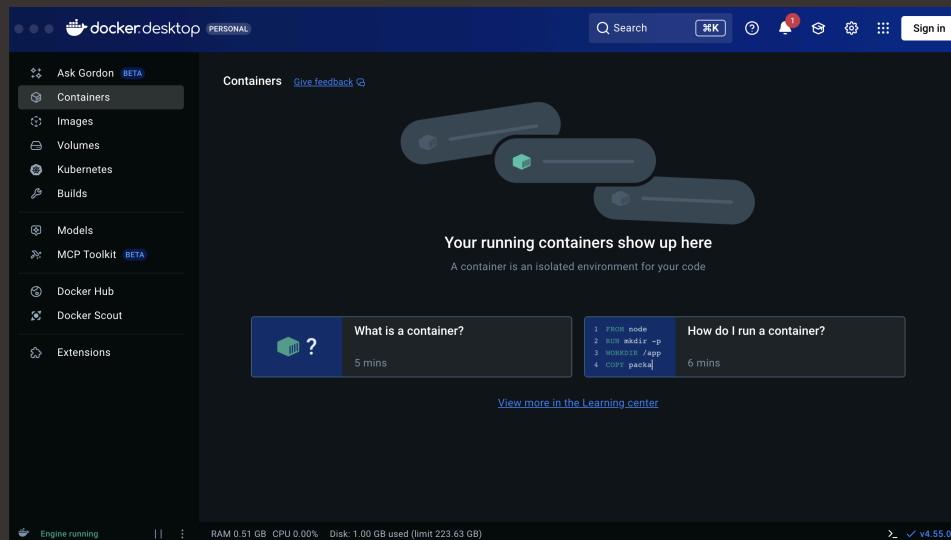


nextwork.org

Deploy an App with Docker



Arpita Chowdhury





Introducing Today's Project!

What is Docker?

Docker is a containerization platform that packages applications and their dependencies into portable containers, and it is useful because it ensures applications run consistently across different environments. I used Docker in this project to build a custom container image, run the application locally, and deploy the same containerized app to AWS Elastic Beanstalk.

One thing I didn't expect...

One thing I didn't expect in this project was how small packaging details like placing the Dockerfile at the root of the ZIP could completely block deployment, showing how precise and detail-oriented real cloud deployments need to be.

This project took me...

This project took me approximately 2 hours to complete. The most challenging part was troubleshooting Elastic Beanstalk deployment errors caused by packaging and networking issues. It was most rewarding to see my custom Docker image successfully deployed and accessible through a live AWS Elastic Beanstalk URL.



Understanding Containers and Docker

Containers

Containers are lightweight, portable packages that bundle an application with all its dependencies and configuration. They are useful because they ensure the application runs consistently across different environments, making development, testing, and cloud deployment more reliable and predictable.

A container image is a read-only blueprint that contains an application's code, required libraries, dependencies, and configuration needed to run it inside a container.

Docker

Docker is a containerization platform that allows applications to be packaged, shipped, and run as containers. Docker Desktop is a local development tool that provides the Docker engine and an easy-to-use interface to build, run, and manage containers on your machine.

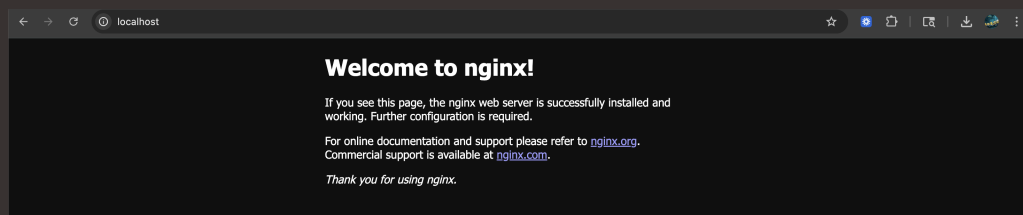
The Docker daemon is the background service that runs on your system and handles building images, running containers, and managing Docker resources by responding to commands sent from the Docker client.



Running an Nginx Image

Nginx is a high-performance web server and reverse proxy that is commonly used to serve web pages, handle HTTP requests, and efficiently deliver content to users.

The command I ran to start a new container was `docker run -d -p 80:80 nginx`, which pulls the Nginx image (if needed), creates a new container, runs it in detached mode, and maps port 80 on my local machine to port 80 inside the container so the web page can be accessed in a browser.



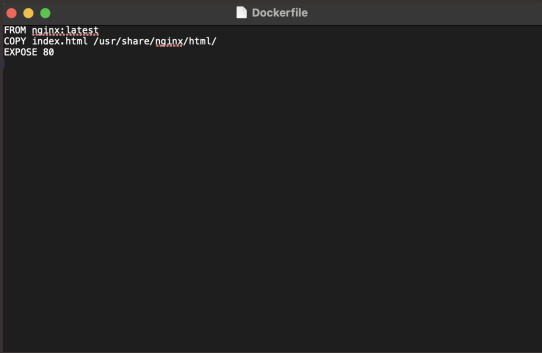


Creating a Custom Image

The Dockerfile is a set of instructions that tells Docker how to build the custom container image by using the Nginx base image, copying my index.html file into the web server's default directory, and exposing port 80 so the application can serve traffic.

My Dockerfile tells Docker three things: to use the Nginx image as the base, to copy my index.html file into Nginx's web directory, and to expose port 80 so the application can receive web traffic.

The command I used to build a custom image with my Dockerfile was `docker build -t my-web-app ..`. The `.` at the end of the command means Docker should use the current directory as the build context, where it looks for the Dockerfile and any files needed during the image build.



```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/
EXPOSE 80
```



Running My Custom Image

There was an error when I ran my custom image because port 80 was already in use by an existing Nginx container, which prevented Docker from binding the same port to the new container. I resolved this by stopping the running container that was already using port 80, then rerunning my container so Docker could successfully map port 80 to my custom image.

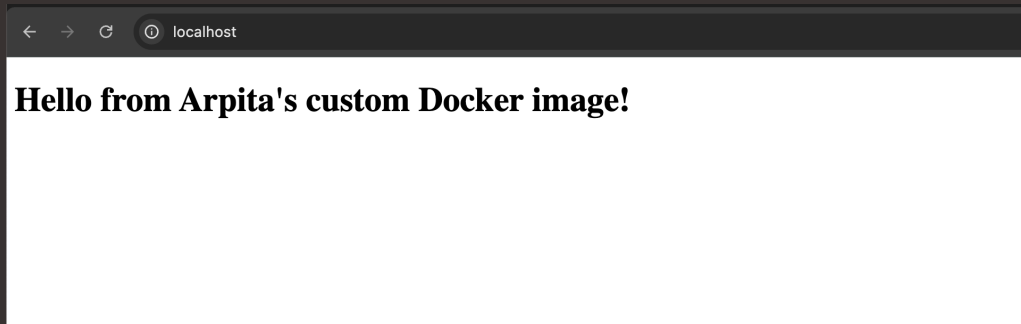
In this example, the container image is my-web-app, which is the blueprint built from the Dockerfile that contains Nginx and my custom index.html. The container is the running instance created from that image, which is what's actively serving the webpage at localhost.



Arpita Chowdhury

NextWork Student

nextwork.org

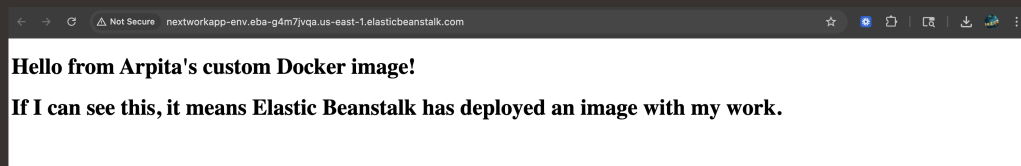




Elastic Beanstalk

Elastic Beanstalk is a fully managed AWS service that lets you deploy and run applications without managing the underlying servers, while automatically handling infrastructure provisioning, scaling, and application availability.

Deploying my custom image with Elastic Beanstalk took me 30 minutes as I encountered with errors and also made a mistake while creating the deployment environment which I had to fix.





Deploying App Updates

To learn how to deploy app updates with Elastic Beanstalk, I updated my app by modifying the index.html file to add new text and an image, then redeploying the updated source bundle. I verified those changes by refreshing the Elastic Beanstalk domain and confirming the new content appeared live in the browser

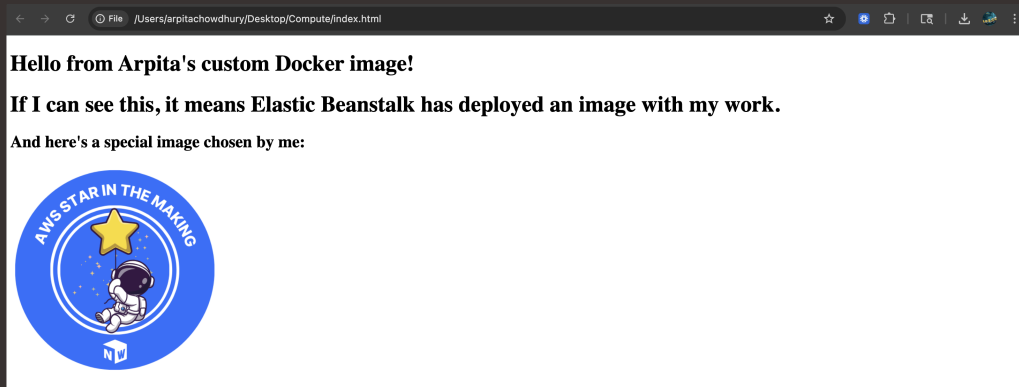
My app updates didn't show up in my live environment immediately because the changes I made to index.html were only saved locally on my computer, while Elastic Beanstalk was still running the previously uploaded ZIP file. To deploy my changes, I only had to re-zip the updated Dockerfile and index.html, upload the new ZIP as a new application version, and deploy it to the existing Elastic Beanstalk environment.



Arpita Chowdhury

NextWork Student

nextwork.org





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

